Which Frames Should We Batch? Bounded-Queue, Value-Driven Frame Admission for Oversubscribed Multi-Camera Edge Inference

Anonymous Authors
Anonymized for review

***Abstract*—Commodity multi-camera rigs—USB webcams on an embedded GPU—lack hardware synchronization and trustworthy timestamps, yet share one detector through a batching stage. First, a measured platform finding: timestamp-based frame alignment there is built on fiction. Decode-stage elements re-stamp frames onto per-camera synthetic grids offset by more than a second, so DeepStream's alignment mode silently discarded 93.6% of frames, while true capture instants already interleave within a median 8.9 ms—tighter than hardware-synchronized nuScenes. The real constraint is GPU time: once the detector outweighs the frame period, not every frame can be processed, and the stock FIFO transport chooses badly and invisibly—at $1.9\times$ oversubscription, 46% of paced frames die in a bounded ring upstream of every pipeline metric, survivors are served through a standing queue set by buffer-pool capacity (276 ms live; up to 855 ms under deep replay pools), and the batcher's deadline knob changes nothing. SPARQ, a bounded-queue scheduler, inverts the drop decision: it stashes at most two frames per camera and, at each inference completion, admits the K most valuable frames by freshness (local arrival clocks only), camera-level activity, and a fairness floor, salvaging recently displaced frames into spare slots. Its output age is pool-independent by construction: 93–146 ms at 85–93% of stock throughput—$2.7\times$ fresher than the live-calibrated baseline, up to $9\times$ under deep pools. FIFO cannot report any event younger than its standing queue; against an independent oracle detector, SPARQ lifts event awareness within 500 ms of onset from 0–19% (pool-dependent) to $\sim$30%, at 0.4% CPU for the scheduler thread. We report equally honest negatives: a saturation defect made our first importance signal inert (we fix it; camera-level importance still yields no recall gain when activity is near-uniform), and salvage recovered $\leq$2% unique detections when objects outlive the inter-service gap.**

***Index Terms*—edge inference, multi-camera perception, dynamic batching, load shedding, DeepStream, real-time systems**

I. Introduction

Multi-camera perception increasingly runs on commodity edge hardware: USB webcams on an embedded GPU sharing one detector for robotics, surveillance, or roadside analytics [1], [2]. Such rigs lack hardware triggers, PTP-disciplined clocks, and trustworthy sensor timestamps: cameras free-run, enumerate sequentially on a shared USB bus, and only the host's arrival clock is reliable. Our platform is deliberately typical: four Logitech C920 webcams (640×480 at 30 fps, MJPG) on one USB-2 bus of an NVIDIA Jetson AGX Orin, running YOLO11 detectors [3] under DeepStream [4].

Sharing one detector forces batching, and batching a dilemma: frames from N cameras are gathered under a deadline, and waiting fills batches and amortizes invocation cost but adds latency to frames already queued [5], [6], [7], [8]. A latency-constrained deployment cannot simply wait longer.

Measuring what the platform actually provides overturns common assumptions. Timestamps downstream of decode are fabricated: GStreamer's `jpegparse` re-stamps frames onto an ideal per-camera 33.33 ms grid anchored at camera startup, staggered USB enumeration offsets the grids by over a second per launch, and timestamp-based alignment (`nvstreammux` with `sync-inputs`) trusted this fiction, silently discarding most frames in our pre-fix campaign. The true capture instants of the four free-running cameras meanwhile interleave far tighter than the 39–46 ms cross-camera spread of hardware-synchronized nuScenes [2]; a scheduler can and should rely on local arrival clocks alone.

The binding constraint is instead GPU time: full-batch YOLO11n inference already takes $\sim$28 ms on the power-capped 612 MHz GPU, the next size up (YOLO11s) sits exactly at the frame period, and any heavier detector oversubscribes the GPU—something must choose which frames die. In the stock pipeline that something is FIFO backpressure, which drops the *newest* frames and serves the stalest: at $1.9\times$ oversubscription (YOLO11m), 46% of paced frames die silently in the transport ring while the pipeline's coverage metric reads 100%, survivors emerge as stale as the transport's buffer pools are deep, and sweeping the batcher's deadline from 5 to 66.7 ms changes none of it. Even without overload, FIFO is measurably unfair: a standing per-camera staleness ladder of 32–241 ms median (frame age at batch push), set by startup order and invisible to throughput metrics. The question is not "how full is the batch?" but: *when not every frame can be processed, which frames should get GPU time?*

We answer with SPARQ, an importance-aware bounded-queue batching scheduler: it sits in front of the batcher, retains at most two frames per camera (the newest plus one displaced-but-interesting held frame), and releases exactly K frames per completed inference, selected by freshness, camera-level activity importance, and fairness. This paper makes three contributions:

- **A measured platform finding.** On commodity USB multi-camera rigs, timestamp-based alignment is fiction: synthetic per-camera timestamp grids are offset by a constant 1.05–1.70 s per launch, and `sync-inputs` silently discards 93.6% of frames, while true capture

instants interleave at a median of 8.9 ms. Alignment is unnecessary *and* harmful; what limits perception under load is which frames get GPU time.

- **SPARQ.** A completion-clocked, content-value-driven admission-and-shedding scheduler for droppable multi-camera frames. It adopts the completion clock of continuous batching in GPU serving [9], [10], [11], but where a serving system must eventually run every request, SPARQ decides which droppable inputs run at all—admission and shedding are one decision. It bounds staleness and the per-camera service interval, and introduces *salvage*: bounded, deferred re-admission of displaced-but-interesting frames into remaining batch capacity—load shedding *with recovery*, a raw-frame analogue of out-of-sequence-measurement processing [12], [13], distinct from classical stream shedding, keep-newest age-of-information policies, CoDel, and fair queueing [14], [15], [16], [17].
- **An honest evaluation.** Experiments on live hardware and on deterministic replay with measured inter-camera offsets injected, scored against an independent oracle detector, with the baseline's queue depth explicitly calibrated: FIFO's standing queue equals its buffer-pool capacity in frames, so its staleness is a *configuration*—276 ms (live pools), 410 ms (live-depth replay), 855 ms (deep replay). SPARQ's output age is pool-independent (93–146 ms mean at 85–93% of stock throughput; p99 139–224 ms)—a 2.7× (live-calibrated) to 9× (deep-pool) freshness gain—and lifts 500 ms event awareness from 0–19% (pool-dependent) to ∼30%; at 2.3× oversubscription its freshness-driven arms win every awareness horizon we measured. We also establish negatives a practitioner needs: the configuration-only keep-newest baseline is inert on this stack; static source-rate decimation either inherits the bufferbloat or sacrifices a quarter of the detections; camera-level importance gains nothing even after we fix a saturation defect that had silently disabled it; and salvage recovers $\leq 2\%$ unique detections when objects outlive the inter-service gap. SPARQ is invariant to injected inter-camera offsets up to 1.7 s.

Section II gives background, Section III the measured pathology, Section IV the scheduler, Section V the evaluation, and Section VI related work; Section VII concludes.

II. Background: Batching on a Commodity Multi-Camera Edge Stack

Our stack is stock DeepStream 7.1 [4] on a Jetson AGX Orin 64 GB (MODE_30W; GPU locked at 612 MHz): four Logitech C920 webcams (640×480 MJPG, 30 fps) share one USB-2 bus, each feeding `v4l2src`, `jpegparse`, and a hardware JPEG decoder into a single shared `nvstreammux`; a YOLO11 [3] FP16 TensorRT detector consumes each batch and a SORT-style [18] NvSORT tracker maintains per-camera identities. The GPU sees only what the mux emits, so every scheduling question reduces to which frames enter the next batch, and when.

Batch formation. The new `nvstreammux` takes two INI rates: $f_{\min}$ (`overall-min-fps`), whose push period $P = 1/f_{\min}$ is the deadline for emitting a (possibly partial) batch, and $f_{\max}$ (`overall-max-fps`). With `sync-inputs` on, a queued frame (running-time PTS B; pipeline time t; $L =$ `max-latency` plus queried upstream latency) is classified[1] as

$$\text{class}(f) = \begin{cases} \text{EARLY}, & B > t - P \\ \text{ONTIME}, & t - P - L \leq B \leq t - P \\ \text{LATE}, & B + L < t - P. \end{cases} \quad (1)$$

EARLY frames wait, ONTIME are batchable, LATE are erased silently—no drop signal fires, so the loss is invisible unless the application counts arrivals. A frame must age P to become batchable, so `sync-inputs` carries a $+P$ latency floor no parameter removes; widening L (*Window A*) moves the drop rate, not latency. A separate co-batching slot (*Window B*) of width $1/f_{\max}$ (8.33 ms at the default $f_{\max}$=120) decides which frames share a batch; sub-frame-period deadlines require raising $f_{\max}$ or the slot throttles the push rate.

The deadline knob that isn't. `nvstreammux`'s `batched-push-timeout` property looks like a direct deadline control, but on DS 7.1 it and the INI key `overall-min-fps` write the *same* internal field, last writer wins.[2] Our harness set the property before loading the INI, so the shipped `min-fps=30` silently overrode every requested timeout across a campaign; an 8-run characterization (deadlines 1–100 ms, with and without an INI) then found the property inert on this code path. The INI's `overall-min-fps` is DS 7.1's only working push-deadline knob (every deadline below is a per-run INI)—and a reproducibility hazard: a sweep varying `batched-push-timeout` on the new mux measures nothing on that axis.

Timestamps do not survive `jpegparse`. `v4l2src` stamps each buffer with the kernel capture time—the only true capture timestamp a UVC camera gives; `jpegparse` discards it, re-stamping onto an ideal grid $\text{first\_pts} + n/\text{framerate}$ anchored at that camera's first frame. Sequential enumeration offsets the grids by the launch's startup stagger (a per-launch constant, 1.05–1.70 s across our launches), so Eq. (1) and everything downstream compare fictitious clocks offset by over a second (quantified in Sec. III). A probe pair around each `jpegparse` restores the truth—a sink probe FIFOs the arriving PTS, a src probe pops it back over the synthetic value (depth ≤ 4, $\sim 1\,\mu$s per probe). Verified live over 120 s: pre-mux PTS equalled the kernel capture stamp on 13 940 of 13 940 batched frames across all four cameras. The fix changes only what downstream elements believe (the sync-off mux is unaffected) and is on in every experiment below.

[1] `NvTimeSync::get_synch_info`, `gstnvtimesynch.cpp:119`; co-batching slot `nvstreammux_batch.cpp:214-231`; silent LATE erasure `nvstreammux_batch.cpp:540-585`; all in the DeepStream 7.1 sources.

[2] `set_batch_push_timeout`, `nvstreammux_batch.cpp:329/333`.

Dynamic-batch TensorRT engines. The engine's dynamic batch dimension (profile batch 1–4) runs a partial batch natively as a smaller invocation rather than padding it: at 612 MHz a single-frame YOLO11n invocation costs about 9 ms of GPU time, a full 4-frame batch $\sim$28 ms (Table I). Batching still amortizes (four frames for roughly $3\times$ the single-frame cost), but a partial batch wastes no work and unused slots admit extra frames at marginal rather than full cost—unlike a static batch-4 engine, which pays the full cost regardless of fill. Low-deadline partial batches are thus cheap, and the spare slots are the capacity SPARQ's salvage mechanism fills.

III. Motivating Measurements

We instrumented the transport layer of the Sec. II rig with pad probes recording kernel capture stamps, pre-mux timestamps, and batch membership. Live runs last 120 s; push-deadline sweeps are deterministic replays injecting inter-camera skew measured live (50 s per point in this campaign's E1; 45 s in the prior F3 sweep; one run per point). Five findings follow.

F1: Synchronization is unnecessary—and harmful—here. The physical premise of alignment is absent: the four free-running cameras interleave their true capture instants with a median spacing of 8.9 ms, tighter than the 39–46 ms RT-BEV reports for the hardware-synchronized nuScenes cameras it aligns [2]. The timestamps are the `jpegparse` fiction of Sec. II: ideal 33.33 ms grids offset per camera by the 1.05–1.70 s enumeration stagger. Fed this fiction, the mux's alignment window (`sync-inputs` =1) silently discarded 93.6% of all frames in our earlier (pre-fix) campaign; in a controlled 120 s live A/B it kept only 14.7% of arrivals (2,059 of 13,985), with camera-biased survival and batching dead by $t \approx 90$ s as synthetic-clock drift (the grid runs $\sim$0.65% fast, accruing $\sim$7 ms of spurious age per second) pushed every frame past the late cut. With true capture timestamps restored, alignment works as documented—99.9% of frames kept, in-batch capture spread p50 2.1 ms—yet still costs the structural one-push-period floor, about $+31$ ms (mean e2e[3] 69.3 ms sync-off vs. 99–102 ms sync-on at the same 33.3 ms cadence), plus $\sim$1% residual discards (coverage 98.9–99.7%). Verdict: run sync-free.

F2: FIFO batching is measurably unfair and stale. Full batches hide a standing queue: in a 120 s live sync-off run with 100% full batches, the true capture spread inside each batch was p50 $\approx$198 ms, and per-camera staleness at batch push formed a 32/172/239/241 ms median ladder set by that launch's camera startup order and permanent thereafter. Fill, throughput, and FPS metrics say the batcher is perfect; freshness says most cameras are served stale, forever.

F3: Waiting buys latency, not fill. Across push deadlines of 1–100 ms, throughput is deadline-invariant (114.6–118.4 frames/s in every stable cell): the deadline moves *when* frames are processed, never *whether*. Beyond one frame period fill saturates—3.85 at a 33.3 ms deadline to 3.96 at 100 ms, $+0.10$ frames of fill while mean e2e rises from 66 to 232 ms—while a dynamic batch-size (1–4) engine rides the deadline the other way, down to 15.3 ms mean e2e at 117.6 single-frame invocations/s with 100% coverage. Batch formation is a pure latency dial.

F4: With a frame-period-matched detector, batching policy is detection-invariant. YOLO11n at batch 4 costs 28 ms on the locked GPU—just under one frame period (ρ=0.84; the knife edge is YOLO11s at 33.5 ms, Table I)—so $\sim$30 full-batch invocations/s absorbs the 4×30 fps input. Here every batching policy we swept produced the same detections: 100.00% frame-matched agreement with the 100 ms-deadline reference (mean $|\Delta|$ 0.0000 detections/frame) in every stable full-coverage cell, flat per-camera rates, track counts 62–63 (agreement is cross-run consistency, not labeled accuracy). The only detection-relevant variable is *coverage*, the fraction of arrived frames processed at all—a null with a sharp implication: while the detector keeps up, no batching policy can matter, and none is needed.

F5: With heavier detectors, the transport layer chooses which frames die—and no deadline setting can intervene. The F4 balance breaks once per-batch service time exceeds the frame period. Table I gives each variant's capacity, and Fig. 1 sweeps the push deadline from 5 to 66.7 ms: for every oversubscribed detector *every* column is flat—fill pinned at 4.00, throughput at capacity, latency at the bufferbloat ceiling. The deadline, the only knob the batching layer offers, is inert; the drop decision was already made upstream. At ρ=1.00 (YOLO11s) no frames are lost, but every result is served through a standing queue converging to $\sim$450 ms in every deadline cell (short-deadline cells report lower whole-run means only because the 50 s runs end before the queue fills)—more than thirteen frame periods stale. At ρ=1.86 (YOLO11m) the pipeline delivers 64 of $\approx$119 paced frames/s (54%) at mean e2e $\approx$850 ms; at ρ=2.33 (YOLO11l), 51 frames/s (43%) at $\approx$1.0 s mean, 1.13 s p99. The missing frames pass through no code that could choose: the mux's queues are unbounded in count, but each queued frame pins a buffer from a finite upstream NVMM pool, so after roughly one pool's worth of standing queue—which fixes the $\sim$850 ms bufferbloat plateau—backpressure reaches the bounded transport ring (the v4l2 kernel ring live; its measured replay stand-in here), which drops the *newest* arrivals upstream of the first instrumented pad. The pipeline's own coverage metric (processed over arrived) reads 100.0% in every overloaded cell while nearly half the paced frames die: under YOLO11m only $\sim$3,200 of $\sim$5,900 paced frames ever *arrive*. Loss is real, frame-blind, camera-arbitrary, and invisible to the system's accounting.

The standing queue obeys an invariant: its depth in *frames*

[3]End-to-end (e2e) latency runs from the source-bin egress pad (local monotonic clock) to tracker output, excluding the upstream transport ring: under FIFO backpressure the ring holds $\sim$4 frames per camera ($\approx$250 ms at YOLO11m rates) that never appear in FIFO's e2e while SPARQ keeps the ring empty, so all FIFO staleness figures here are *lower bounds* and the comparison conservative. e2e also understates glass-to-out by the 20–28 ms a frame ages in-camera and on USB; sweep values are replay-based and exclude one MJPEG decode stage ($\sim$25 ms) present live.

TABLE I

MEASURED DETECTOR CAPACITY ON THE 612 MHZ-LOCKED ORIN GPU (FP16, BATCH 1–4 DYNAMIC ENGINES, 640×480). $S(4)$ FOR SATURATED MODELS IS DERIVED FROM SATURATED THROUGHPUT (4000/F/S), WHICH IS QUEUE-FREE; ARRIVAL LOAD IS $4 \times 29.8 \approx 119$ FRAMES/S.

detector	$S(1)$ (ms)	$S(4)$ (ms)	capacity (f/s)	load ρ
YOLO11n	8.8	28.3	141	0.84
YOLO11s	–	33.5	119	1.00
YOLO11m	–	62.4	64	1.86
YOLO11l	–	78.3	51	2.33

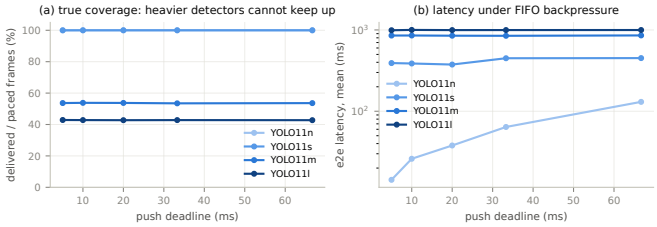

Fig. 1. The capacity wall (E1). (a) Delivered throughput as a share of paced input vs. push deadline: heavier detectors cannot keep up, and the deadline does not help (YOLO11n and YOLO11s overlap at 100%). (b) Mean e2e latency: oversubscribed detectors serve every result through a standing queue regardless of deadline; YOLO11s holds 100% delivery only by serving everything ∼400 ms stale. One 50 s deterministic replay per point.

is the transport's buffer-pool capacity, independent of detector. Across YOLO11s/m/l the measured backlog is 51–55 frames under this replay configuration, so staleness is pool over capacity: $55/64.5 = 852$ ms (m), $54/119 = 449$ ms (s), $51.5/46.5 = 1108$ ms (l), each matching the measured plateau. FIFO's staleness is thus a *configuration*, not a constant—the evaluation (§V) varies pool depth explicitly and calibrates to the live rig's shallower pools.

Together these findings pose the question this paper answers: *when not every frame can be processed, which frames should be?* They also fix the design envelope. Frame value must come from local arrival clocks (`CLOCK_MONOTONIC` at arrival), never from stream PTS, which F1 shows can be synthetic and per-camera offset by up to ∼1.7 s; and the scheduler must run with `sync-inputs` =0, replacing alignment rather than stacking on it. F2–F4 identify the levers: FIFO order and batch-formation waiting are free to change, since neither buys detection quality; what matters under overload is admission—which frames get GPU time.

IV. SPARQ DESIGN

When batch service time exceeds the frame period, FIFO backpressure sheds the *newest* frames and serves the stalest—maximal staleness, arbitrary per-camera loss, bufferbloat latency. SPARQ inverts this: a bounded per-camera stash between sources and batcher sheds frames by default, and a scheduler decides which earn GPU time. Every drop is counted: arrivals equal admissions plus policy drops exactly on every instrumented run.

A. Principle: Value-Driven Admission at the Completion Clock

Each camera c stashes at most two frames: $fresh[c]$, the newest arrival, and $held[c]$, a displaced frame retained by the salvage rule (§IV-C); all other arrivals drop immediately, so queues cannot grow. A scheduler thread, clocked by inference completions, releases the K most valuable stashed frames across all cameras to `nvstreammux` as one batch.

Completion-clocked release is adopted from continuous batching in model serving [9], [10], [11], [6], [8], [7]; the delta is what the clock decides. Serving must eventually run every admitted request, choosing only when and with whom; saturated camera frames are droppable, so SPARQ chooses *which inputs run at all*—admission and load shedding fused into one last-moment decision. All timing uses local arrival clocks (`CLOCK_MONOTONIC` stamped at interception), never stream PTS.

B. Value Function and Fairness Floor

Candidates are ranked by a value with all terms normalized to $[0, 1]$:

$$\mathrm{fresh}(f) = \max\big(0,\, 1 - \mathrm{age}(f)/\tau_{\max}\big), \tag{2}$$

$$\mathrm{imp}(c) = I_c / I_{\max}, \tag{3}$$

$$\mathrm{fair}(c) = \min\big(1,\, (t_{\mathrm{now}} - t_{\mathrm{srv}}(c))/D_{\mathrm{fair}}\big), \tag{4}$$

$$v(f) = w_f\,\mathrm{fresh}(f) + w_i\,\mathrm{imp}(c_f) + w_r\,\mathrm{fair}(c_f), \tag{5}$$

where $\mathrm{age}(f) = t_{\mathrm{now}} - t_{\mathrm{arr}}(f)$, $t_{\mathrm{srv}}(c)$ is camera c's last service instant, and $D_{\mathrm{fair}} = 2\,(N/K)\,\hat{S}(K)$ with $\hat{S}(K)$ an online EWMA of batch-K service time. Importance I_c is a tracker-probe EWMA (half-life 2 s) of *new-track events* on camera c, clipped to $[0, I_{\max}{=}2]$. The signal must measure change, not content: a first formulation with a standing-detection term ($3 \cdot$ new tracks + detections, $I_{\max}{=}10$) saturated on any persistent-object scene—measured median per-admission score 1.000 on *every* camera, silently reducing `imp` to `fresh`. The evaluation uses the event-driven form, which tracks oracle event counts (per-camera medians 0.22–0.66); it measures camera-level activity, not frame-level semantics—the scheduler never inspects unprocessed pixels, which no model has seen. Defaults (fixed throughout the evaluation): $w_f{=}0.4$, $w_i{=}0.35$, $w_r{=}0.25$; $\tau_{\max}{=}150$ ms (older fresh frames are evicted, so no *admitted* frame exceeds that age; service time adds on top). A weight-sensitivity sweep is future work, mitigated by the ablation ladder, whose arms bound the weights' effect from both sides ($w_i{=}0$ vs. the full function).

Since fair only biases the ranking, a hard floor force-admits any camera with $t_{\mathrm{now}} - t_{\mathrm{srv}}(c) > D_{\mathrm{hard}} = 4\,D_{\mathrm{fair}}$ into the next release. *Property (bounded service interval).* While a camera delivers frames *and* at least K of N cameras remain live (full K-releases keep forming), its service interval never exceeds $D_{\mathrm{hard}} + \lceil N/K \rceil\, S(K)$: crossing D_{hard} forces inclusion within $\lceil N/K \rceil$ releases (several completions if more than K cameras cross at once), and completions under load arrive at most one batch service time apart. $\hat{S}$ is measured release-to-completion at pipeline depth 2 (≈1.9× the raw batch service time), so

$D_{\text{hard}} \approx 1.25\,\text{s}$ at YOLO11m/K=2. Measured post-warmup service gaps respect the bound in replay (max 0.38 s) and live `imp` runs (max 0.84 s); the live salvage arm grazed it twice (by $\leq$1.3%). Starvation of a live camera is impossible by construction, in the spirit of weakly-hard guarantees [19], [20]. Camera failure breaks the $N \geq K$ assumption only at $K=N$; at $K<N$ the scheduler tolerates $N-K$ dead cameras unchanged, and the general fix (a timed partial release) is future work.

C. Selection Pressure and Salvage

Batch size K is the policy knob. $K=N$ admits every fresh candidate and makes importance inert—the degenerate all-admit case, kept as an ablation arm. At $K<N$ (default K=2 of N=4) cameras compete for slots, and the value function allocates the GPU's batch rate: active cameras are served more often, quiet ones at least every $D_{\text{hard}} + S(K)$.

Salvage extends admission one frame backward: a frame displaced unserved is retained as $held[c]$ iff $\text{imp}(c) \geq 0.3$ at displacement; held frames older than τ_{salv}=250 ms are evicted, bounding memory ($\leq 2N$ buffers) and recovery depth. Held frames compete for the same K slots, admitted only when outranking a fresh candidate; a hot camera may take two slots (fresh+held, via `max-same-source-frames` = 2). One mechanical cost: a held frame admitted in a *later* release than its displacing sibling reaches the tracker with a per-camera timestamp regression (measured on 1.4–6.4% of salvage-mode emissions; zero in all other modes), which fragments tracks (§V). This is load shedding [14] *with bounded recovery*: it lifts the out-of-sequence-measurement principle—late data still carries value and should be incorporated, not discarded [12], [13]—from tracking's measurement level to raw frames, before inference. Salvage adds no batch-formation wait (release still follows the completion clock); its service and live-path latency costs are measured, not assumed (+14 ms mean e2e and +20 ms p99 over `imp` at YOLO11m; §V).

D. Mechanism on DeepStream

SPARQ modifies no DeepStream plugin. A buffer probe on each source bin's output pad returns `DROP` while taking a reference—upstream sees successful delivery; the frame joins the stash—and a thread-local guard passes the scheduler's own pushes. The event-driven scheduler thread (condition variable signaled by arrival and completion probes; 5 ms fallback timer) waits for in-flight work below $(depth-1)K$ frames (double buffering, $depth$=2) and $\geq K$ candidates, then evicts stale frames, applies the fairness floor, and pushes the top-K in ascending per-camera PTS order.

For batch atomicity, `nvstreammux` and the detector both run at batch size K: the mux's readiness check completes on the Kth buffer, emitting one batch immediately. One configuration trap: on the new `nvstreammux` the push deadline lives in the config file's `overall-min-fps` (`batched-push-timeout` aliases the same field, last writer wins), and adaptive batching silently overrides the batch-size property with the source count. SPARQ therefore disables adaptive batching and sets both mux rate anchors slower than any service period, so the mux clock never preempts a forming K-burst; measured, 99.9% of batches then carry exactly K frames. In-flight work is counted in *frames* ($+K$ on release, $-$`num_frames_in_batch` at the tracker-side probe, clamped at zero with a watchdog), so an unexpected batch split cannot corrupt the completion clock. EOS passes through untouched (the stock teardown path) and the scheduler releases that camera's stash; at most two tail frames per camera go unprocessed, immaterial for steady-state measurements.

Four modes form the ablation contract: `off` (probes not attached; bit-identical to the unmodified pipeline), `fresh` (w_i=0: keep-newest with fairness), `imp` (the full value function), and `salvage` (`imp` plus held slots)—each adds exactly one ingredient, so the evaluation can attribute every effect. The scheduler is ~600 lines of C++ against DeepStream 7.1's new `nvstreammux` [4]; its thread costs 0.40% of one CPU core at the primary operating point (§V).

V. Evaluation

A. Setup

All experiments use the rig of Section III: Jetson AGX Orin 64 GB in `MODE_30W`, clocks locked (GPU 612 MHz, CPU 1.73 GHz), DeepStream 7.1, FP16 dynamic-batch (1–4) TensorRT engines at 640×480, NvSORT tracker, `sync-inputs` =0.

Workload. Deterministic replay of four ~60 s clips from the four live cameras, injecting the rig's measured timing imperfections: startup stagger (0/1134.8/1702.1/567.2 ms), per-camera clock-rate factors (≈0.961, the C920's true 32.0 ms cadence), periodic 2-frame capture gaps, and a 4-deep drop-newest ring emulating the v4l2 kernel ring. Frames are keyed by (camera, capture timestamp), so policies compare frame-for-frame—impossible live; live runs validate transfer (§V-D).

Baseline queue-depth calibration. FIFO's standing queue equals its buffer-pool capacity (Sec. III F5), so the baseline runs at two depths: the replay default (20 decoder surfaces per camera; backlog 51–55 frames, e2e ~855 ms at YOLO11m) and a *live-depth* configuration (2 surfaces; backlog ~24 frames, e2e ~410 ms, stationary) bracketing the live rig's measured 276 ms from above; intermediate depths sit in a slowly-filling non-stationary regime and are excluded. SPARQ never fills the pools; its numbers are depth-independent.

Arms. Ten at the primary detector: **FIFO-33** (stock pipeline, 33.3 ms push deadline, deep pools), **FIFO-33-LD** (live-depth pools), **FIFO-5** (5 ms deadline), **DROP-OLD** (configuration-only keep-newest: per-camera 1-deep `leaky=downstream` queue), **DEC-1/2** and **DEC-1/3** (static source-side decimation to ρ=0.93 / 0.62), **FRESH-K4** and **FRESH-K2** (SPARQ, w_i=0), **IMP-K2** (full value function, event-driven importance per §IV), and **SALV-K2** (adding salvage slots). Five repeats for deep-pool stock arms and FRESH, three elsewhere and at YOLO11s (ρ=1.00) and YOLO11l (ρ=2.33). Drop patterns vary with thread timing (same-arm repeats share only ~43% of processed frames), so we report medians with *min–max ranges* (bootstrap degenerates at $n \leq 5$).

TABLE II

POLICY COMPARISON AT YOLO11M (ρ=1.86), MEDIANS OVER REPEATS (TEXT WHISKERS ARE MIN–MAX). COVERAGE IS OVER THE WHOLE-CLIP ORACLE FRAME SET; TTA@Δ = EVENTS (OF 123) RECALLED WITHIN Δ OF ONSET IN EMISSION TIME. FIFO/DROP-OLD LOSE IN THE TRANSPORT RING, DEC AT THE SOURCE; SCHEDULER LOSSES ARE CHOSEN BY POLICY.

policy	coverage (%)	e2e mean (ms)	e2e p99 (ms)	det. yield (%)	TTA@0.5s (%)
FIFO-33	46.2	857	1032	34.2	0.0
FIFO-5	46.3	853	1016	34.2	0.0
DROP-OLD	46.1	856	1006	34.3	0.0
FIFO-33-LD	44.8	410	601	37.6	18.7
DEC-1/2	43.1	997	1265	31.6	0.0
DEC-1/3	29.9	64	123	21.5	30.1
FRESH-K4	41.1	146	224	30.4	30.9
FRESH-K2	38.8	93	139	28.6	30.1
IMP-K2	31.7	115	200	23.3	28.5
SALV-K2	31.7	129	220	25.1	24.4
	34.1	577			

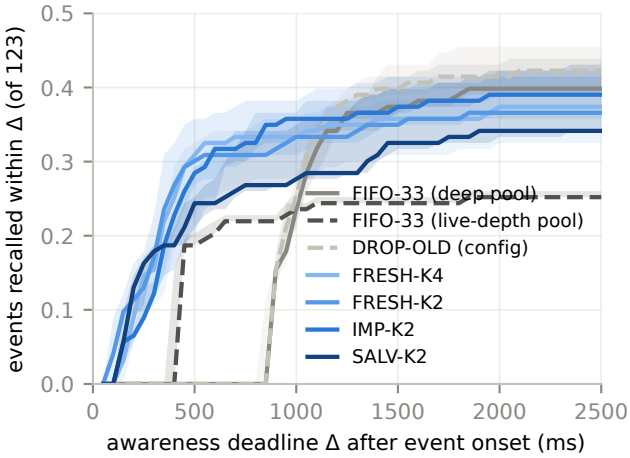

Fig. 2. Time-to-awareness recall at YOLO11m: fraction of the 123 oracle events with a matching detection *emitted* within Δ of onset (median, min–max band over repeats). FIFO cannot report events younger than its standing queue (855 ms deep-pool, ∼410 ms live-depth); SPARQ's bounded queue sits at the far left; live-depth FIFO plateaus low (its pool-starved pacer loses whole event windows).

Oracle and metrics. To avoid circularity, a strictly larger detector (YOLO11x) processes *every* frame in an offline completeness pass (ring disabled; backpressure pauses the pacer). Oracle detections (conf. $\geq$0.4) are greedily IoU-tracked per camera and class; an *event* is a new oracle object persisting $\geq$3 oracle frames: 123 total, whose composition we disclose because a detector oracle has failure modes of its own—63 *clean* first appearances, 39 re-detections of static objects after confidence dips (laptop, TV), 21 office-implausible classes (YOLO flicker). We report recall on all 123 and on the 63 clean; person-only (n=14) is similar but too small to separate arms. *Coverage*: frames processed over the whole-clip oracle frame set (a lossless 50 s run caps at ∼86%; within its own window the stock deep-pool pipeline delivers 54% of paced frames, matching F5 and live; relative comparisons shift <0.5%). *Detection yield*: oracle detections matched (class + IoU$\geq$0.3) over all oracle detections. *Time-to-awareness (TTA) event recall*: an event is recalled within Δ if a matching detection is *emitted* within Δ of onset (emission time = onset delay + run-mean e2e); validated against exact per-record emission stamps and a full-distribution convolution, accurate to $\pm$0.02 at every Δ, exact below 500 ms for deep-pool FIFO (p5 e2e 722 ms). Latency (e2e): per-batch metrics, 5 s warmup trimmed, stamped post-transport-ring—understating FIFO staleness by up to ∼250 ms and SPARQ's not at all (footnote, §III), so the comparison is conservative.

B. Policy Comparison (E2)

Table II and Fig. 2 give the YOLO11m results; Table III the other two load points.

The staleness inversion. The deep-pool stock arms are indistinguishable: FIFO-33, FIFO-5, and DROP-OLD all deliver 46% coverage at ∼855 ms mean e2e (p99 1.0–1.03 s). The deadline knob does nothing (F5), and DROP-OLD is *inert*: the mux's count-unbounded internal queues never backpressure its 1-deep leaky queue—the queue it would need to drain forms downstream of it. Shrinking pools helps but cannot catch up: FIFO-33-LD serves at 410 ms (p99 601 ms) and loses throughput to pool starvation (58 vs. 64.5 frames/s). SPARQ inverts the pathology at any depth: FRESH-K4 delivers 41% coverage at 146 ms (93% of deep-pool FIFO's throughput, a 5-point coverage cost), FRESH-K2 39% at 93 ms, p99 139–224 ms. Freshness gains: 2.8–4.4× vs. the live-calibrated baseline, 5.9–9× vs. deep pools—wider still per-frame, since batch-max e2e overstates the scheduler's.

Static decimation is not a substitute. The practitioner remedy—decimate each source until the detector keeps up—depends brutally on the ratio. Just below capacity (DEC-1/2, ρ=0.93), the startup backlog never drains: 997 ms mean e2e, flat across the run, the *stalest* arm measured, zero sub-second event awareness. Conservative (DEC-1/3, ρ=0.62) is competitively fresh (64 ms mean, 123 ms p99) and ties the scheduler at TTA@500 ms—but processes and detects roughly a quarter less (coverage 30% vs. 39%, yield 21.5% vs. 28.6% for FRESH-K2), must be re-tuned per detector and load, and bounds neither staleness nor capacity reallocation under drift. SPARQ is decimation made dynamic, load-proportional, and value-ordered.

Event awareness. No FIFO configuration reports an event younger than its standing queue: deep-pool FIFO recalls *zero* events within 500 ms (p5 e2e 722 ms), live-depth FIFO 18.7%, SPARQ arms 24–31%; on the 63 clean events the separation widens—23.8% (FIFO-LD) vs. 38–43% (SPARQ), deep-pool still zero. Past ∼0.9–1.2 s the deep-pool curve crosses: its coverage advantage recalls more events *eventually* (0.40 vs. 0.39 for IMP-K2 at 2 s; both 0.54 on clean events), while live-depth FIFO never catches up (0.25 at 2 s), its pool-starved pacer dropping correlated bursts and missing whole event windows. On this measured freshness–coverage frontier, every stock configuration is dominated under a sub-second awareness requirement.

TABLE III
POLICY COMPARISON AT THE OTHER TWO LOAD POINTS (MEDIANS; WHOLE-CLIP ORACLE-FRAME COVERAGE AS IN TABLE II; DEEP-POOL REPLAY; IMP/SALV ROWS USE THE V1 SATURATED SIGNAL AND ARE THEREFORE FRESH-EQUIVALENT AT THESE LOAD POINTS).

policy	coverage (%)	e2e (ms)	p99 (ms)	yield (%)	TTA@0.5s (%)	TTA@2s (%)
YOLO11s, $\rho = 1.00$ (3 repeats)						
FIFO-33	78.6	497	609	62.5	22.8	39.8
FRESH-K4	60.1	97	162	47.1	32.5	37.4
FRESH-K2	69.3	58	78	55.2	35.8	41.5
IMP-K2	68.4	60	89	54.5	35.8	39.8
SALV-K2	65.3	71	154	53.4	33.3	38.2
YOLO11l, $\rho = 2.33$ (3 repeats)						
FIFO-33	33.7	1112	1280	24.6	0.0	30.9
FRESH-K4	30.8	192	272	22.6	21.1	34.1
FRESH-K2	28.8	121	186	20.7	21.1	32.5
IMP-K2	28.7	122	187	20.9	19.5	32.5
SALV-K2	29.0	128	168	20.8	21.1	25.2

Importance: a saturation defect, fixed, and an honest null. Our first importance signal (weighted new-tracks + standing detections, clipped EWMA) silently saturated: measured across all admissions, median score 1.000 on every camera, 68% of decisions at $\geq$0.99—so `imp` mode had been *structurally identical* to `fresh`, its apparent per-camera differences mere selection dynamics. We report this defect because persistent-object scenes will saturate any content-magnitude signal: a practitioner trap. With the event-driven signal (§IV; per-camera score medians 0.22–0.66, ordered by oracle event counts, 1% saturated), IMP-K2 concentrates service on event-active cameras, paying in throughput (46.6 vs. 54 frames/s for FRESH-K2, from more single-camera-heavy selection) and coverage (31.7% vs. 38.8%)—yet event recall does not improve (TTA@500 ms 28.5% vs. 30.1%, within min–max overlap; clean events 38% vs. 43%). The null is now genuine, and the oracle shows why: events split 37/33/28/25 across cameras, leaving camera-level reallocation little to win. Strongly skewed workloads remain future work.

Salvage: mechanically sound, $\leq$2% unique. SALV-K2 re-admitted $\sim$580 displaced frames per run at +14 ms mean e2e over IMP-K2, yet of 435 oracle-matched detections on salvaged frames in an instrumented run, none were unique within a $\pm$150 ms neighbor-coverage window and 9 (2.1%) within a window equal to the median inter-service gap ($\sim$70 ms): where objects persist for seconds, the next fresh frame carries the recovered information anyway. Salvage also costs—cross-release re-admissions reach the tracker with per-camera timestamp regressions (1.4–6.4% of its emissions; zero in every other mode), measurably fragmenting tracks (§V-D)—and its recall trails IMP-K2. We claim no utility for salvage on this workload; its regime, objects visible shorter than the inter-service gap, did not occur here.

Across the load range (Table III; deep-pool baselines). At ρ=1.00 (YOLO11s) the stock pipeline drops almost nothing (coverage 79%) but its queue converges to $\sim$450 ms; FRESH-K2 chooses 69% coverage at 58 ms (7.8$\times$ fresher) and lifts TTA@500 ms from 0.23 to 0.36. At ρ=2.33 (YOLO11l) the coverage gap nearly closes (34% FIFO vs. 29–31% SPARQ) while latency splits 1.1 s vs. 121–192 ms, and the freshness-driven arms (FRESH-K4/K2, IMP-K2) win *every* TTA horizon including 2 s (0.33–0.34 vs. 0.31; SALV-K2 trails at 0.25): FIFO's eventual recall arrives too late to count even at generous deadlines. The heavier the overload, the more the freshness-driven scheduler dominates.

C. Offset Robustness (E3)

SPARQ's clocks are local by construction; we verify by scaling the injected inter-camera stagger from 0 to the measured live maximum (1.7 s) at YOLO11m. IMP-K2 is flat: 44.5–50.8 frames/s, e2e 100–107 ms across all offsets (run-to-run noise, no trend). The timestamp-trusting configuration (`sync-inputs` =1 on fabricated timestamps, F1's pre-fix stock world) processes only 25–36 frames/s at $\sim$110–265 ms at *every* offset including zero (one zero-offset repeat degenerated to 11 frames/s outright): its loss is dominated by synthetic-clock drift rather than the constant offset, consistent with F1's live finding. No timestamp quality, at any offset, affects SPARQ.

D. Live Validation (E4)

Three 120 s runs on the four physical cameras (YOLO11m) confirm transfer. Under FIFO-33 only 6,433 of $\sim$14,000 captured frames ever *reach* the pipeline—the kernel ring silently discards 54%—and survivors are served at 276 ms mean (p99 518 ms; live pools are shallower still than the LD replay configuration). IMP-K2 receives *all* 13,984 frames (arrival matches the cameras' measured delivery within 1% in every 10 s window—no detectable ring loss) and processes 50.1 frames/s—92% of FIFO's live throughput—at 101 ms mean / 155 ms p99: 2.7$\times$ fresher, every drop a counted policy decision. SALV-K2 live shows the tracker cost of its timestamp regressions: 79 distinct track IDs vs. 55 (IMP) and 44 (FIFO), measurable fragmentation consistent with its replay null.

E. Overheads and Measurement Costs

The scheduler thread costs 0.40% of one CPU core at YOLO11m (whole app: 23%); the stash holds at most $2N$=8 NVMM buffer references; drop accounting closes exactly (arrivals = admissions + policy drops on every instrumented run). Three honest costs remain. First, the completion-clocked release cycle adds $\sim$5–10 ms per batch—a throughput tax (FRESH-K4 vs. deep-pool FIFO-33) of 20% at YOLO11s, 7% at YOLO11m, 4% at YOLO11l, shrinking as service time grows. Second, small batches change what the detector finds: matched frame-for-frame against a same-model completeness pass, batch-2 arms reproduce 90–93% of the reference detections on the frames they process (FIFO's batch-4 arms: 96–100%)—an FP16/batch-size numerics effect, distinct from scheduling, that accounts for part of the yield gap. (Against that same-model reference, yields are 45% for deep-pool FIFO and 31–35% for the K2 arms; Table II's YOLO11x-oracle numbers are uniformly $\sim$27 points lower because YOLO11m

itself matches only $\sim$73% of x's detections even on processed frames.) Third, at light load (YOLO11n, ρ=0.84) the always-K release quantizes admission: 96.6% coverage vs. the stock pipeline's 100% (K=2), against 24 ms vs. $\sim$64–70 ms mean e2e (warmup-trimmed). A knowingly underloaded deployment should run the stock path (`-sched off` is bit-identical to it); SPARQ is the overload regime's tool.

VI. Related Work

a) Streaming perception: Streaming perception charges a detector for its own latency, scoring accuracy against the world at output time [21]; the ensuing line trades coverage for freshness (frame skipping, forecasting to output time, runtime adaptation of model scale and execution) [22], [23], [24], [25], [26], as do deadline-driven autonomous-driving pipelines [27], [28]—the right trade for control, where a stale detection is a liability. SPARQ targets the complementary coverage-critical monitoring regime, where a missed event is never recovered: rather than discarding whatever is late, it decides which droppable frames still deserve GPU time and bounds their staleness. Criticality-aware attention scheduling [29] also treats pixels unequally but partitions one camera's field of view; SPARQ arbitrates whole frames across cameras.

b) Out-of-sequence measurements: Tracking has long refused to discard late data: out-of-sequence-measurement (OOSM) filters retrodict state to fold in delayed measurements [12], [30], and automotive fusion weighs buffering against such algorithms [13]—all post-detection, at the measurement level. SPARQ lifts retention upstream to raw frames: a displaced frame from a recently active camera is briefly held and re-admitted into unused batch capacity (salvage) before it becomes a missing measurement.

c) Inference serving and batching: Batching under latency objectives is model serving's core mechanism: timeout-bounded adaptive batching [5], [7], predictability-first scheduling [11], SLO-aware batch sizing [6], multi-entry multi-exit batches [8], iteration-clocked LLM batching [9], [10], edge co-adaptation of batch size with model choice or concurrency [31], [32]. SPARQ adopts the completion-clocked discipline but not the input contract: a serving system must eventually answer every request, so its levers are ordering and sizing; camera frames are perishable and droppable, so SPARQ fuses admission and shedding—choosing which frames run at all *is* the scheduling problem, not an error path.

d) Load shedding, queue management, and freshness: Each ingredient of that decision has a lineage: semantic load shedding in stream databases [14], [33], CoDel's delay- rather than length-bounded queues [16], deficit-round-robin fairness [17], LIFO-flavored overload admission [34], age-of-information results that small keep-newest queues minimize staleness [15], [35], and imprecise-computation and weakly-hard (m, k) models of acceptable partial or skipped work [36], [37], [20], [19]. SPARQ deliberately composes these known scaffolds (keep-newest per camera, value-ranked admission, a DRR-flavored fairness floor with a weakly-hard-style per-camera service-interval bound); novel are salvage—bounded deferred recovery of already-shed frames into spare batch capacity, load shedding with a second chance, which none of the above provides—and a semantic value function over cameras rather than tuples or packets.

e) Video analytics and multi-camera edge perception: Video-analytics systems filter frames by content to save downstream compute: cheap on-camera features [38], profile-driven configuration adaptation [39], model cascades [40], learned frame skipping [41]—deciding *which* frames merit analysis; SPARQ additionally decides *when* they run through a shared, saturated batching stage, and can revisit a drop. Among multi-camera edge systems, REVAMP^{2}T distributes pedestrian re-identification across Jetson-class nodes [1]; RT-BEV co-optimizes camera synchronization with region-of-interest-aware fused bird's-eye-view perception [2]. RT-BEV's align-before-fusion premise is sound for hardware-synchronized nuScenes keyframes on a GPU desktop (RTX 3080), but our measurements show it inverts on a commodity USB rig: four free-running cameras' true capture instants already interleave within a few milliseconds at the median, the stock pipeline's timestamp-based alignment silently discards most frames, and per-camera detection needs no alignment. SPARQ treats synchronization as a constraint to shed rather than a primitive to enforce, requiring only local arrival clocks.

VII. Conclusion

Commodity multi-camera perception does not fail where the literature assumes. Alignment is already solved by physics on a shared USB rig (true capture interleaving of 8.9 ms median) and actively sabotaged by fabricated timestamps: we measured DeepStream's alignment mode discarding 93.6% of frames because of a decode-stage re-stamping defect, and provide the fix. The binding problem is admission: once the detector outweighs the frame period, the stock pipeline lets transport backpressure decide which frames die—newest first, invisibly, its coverage metric reading 100%—serves survivors hundreds of milliseconds stale, and its batching deadline knob is measurably inert (and aliased in source). SPARQ makes that decision explicit: a bounded per-camera stash, value-driven admission at the inference completion clock, a hard staleness bound and per-camera service-interval guarantee, and salvage of displaced-but-interesting frames into remaining batch capacity—load shedding with bounded recovery. Measured at 1.9$\times$ oversubscription, that inversion makes output age pool-independent—2.7$\times$ fresher than the live-calibrated stock pipeline and up to 9$\times$ under deep pools, for 5–7 points of coverage at 0.4% CPU—lifting 500 ms event awareness from 0–19% (set by the baseline's pool depth) to $\sim$30%, the advantage widening with load until the freshness-driven arms win every awareness horizon we measured. Beyond the scheduler, deterministic offset-injected replay keyed by capture timestamps and scored against an independent oracle detector lets frame-level scheduling decisions be audited exactly; the platform findings (timestamp fiction, inert deadline knob, metric-invisible transport loss) reproduce on stock

DeepStream 7.1, and the released code, configurations, and analysis pipeline reproduce every number.[4]

Limitations: importance is camera-level and lagging (an event on a long-quiet camera surfaces only at the fairness-floor cadence); the bounded-service guarantee assumes at least K live cameras—camera failure at $K{=}N$ needs a timed partial-release fallback not yet implemented; salvage recovers real frames but of honestly small marginal value in slowly-changing scenes ($\leq 2\%$ unique), and its cross-release re-admissions fragment tracks; event ground truth comes from a detector oracle whose raw event set is itself noisy (we stratify and report clean events separately); and all results are from one rig class (USB-2 webcams, one embedded GPU). Control-critical settings needing predict-forward freshness remain the domain of streaming perception.

REFERENCES

[1] C. Neff, M. Mendieta, S. Mohan, M. Baharani, S. Rogers, and H. Tabkhi, "REVAMP^{2}T: Real-time edge video analytics for multicamera privacy-aware pedestrian tracking," *IEEE Internet of Things Journal*, vol. 7, no. 4, pp. 2591–2602, Apr. 2020.

[2] L. Liu, J. Lee, and K. G. Shin, "RT-BEV: Enhancing real-time BEV perception for autonomous vehicles," in *2024 IEEE Real-Time Systems Symposium (RTSS)*. York, UK: IEEE, 2024, pp. 267–279.

[3] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] NVIDIA Corporation, "NVIDIA DeepStream SDK," https://developer.nvidia.com/deepstream-sdk, 2024, version 7.1, released October 2024. Developer guide: https://docs.nvidia.com/metropolis/deepstream/dev-guide/. Accessed 2026-07-08.

[5] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. Boston, MA: USENIX Association, 2017, pp. 613–627. [Online]. Available: https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw

[6] H. Shen, L. Chen, Y. Jin, L. Zhao, B. Kong, M. Philipose, A. Krishnamurthy, and R. Sundaram, "Nexus: A GPU cluster engine for accelerating DNN-based video analysis," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*. Huntsville, ON, Canada: ACM, 2019, pp. 322–337.

[7] NVIDIA Corporation, "Triton inference server: An optimized cloud and edge inferencing solution," https://github.com/triton-inference-server/server, 2019, dynamic batching documentation: https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/batcher.html. Accessed: 2026-07-08.

[8] W. Cui, H. Zhao, Q. Chen, H. Wei, Z. Li, D. Zeng, C. Li, and M. Guo, "DVABatch: Diversity-aware multi-entry multi-exit batching for efficient processing of DNN services on GPUs," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, 2022, pp. 183–198. [Online]. Available: https://www.usenix.org/conference/atc22/presentation/cui

[9] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Koblenz, Germany: ACM, Oct. 2023, pp. 611–626.

[4] Artifact link withheld for review.

[11] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like clockwork: Performance predictability from the bottom up," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 443–462. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/gujarati

[12] Y. Bar-Shalom, H. Chen, and M. Mallick, "One-step solution for the multistep out-of-sequence-measurement problem in tracking," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 40, no. 1, pp. 27–37, Jan. 2004.

[13] M. Mauthner, W. Elmenreich, A. Kirchner, and D. Boesel, "Out-of-sequence measurements treatment in sensor fusion applications: Buffering versus advanced algorithms," in *Workshop Fahrerassistenzsysteme (FAS 2006)*, Löwenstein/Hösslinsülz, Germany, 2006, pp. 20–30. [Online]. Available: https://mobile.aau.at/~welmenre/papers/mauthner-fas06.pdf

[14] N. Tatbul, U. Çetintemel, S. B. Zdonik, M. Cherniack, and M. Stonebraker, "Load shedding in a data stream manager," in *Proceedings of the 29th International Conference on Very Large Data Bases (VLDB '03)*. Berlin, Germany: VLDB Endowment, 2003, pp. 309–320.

[15] S. Kaul, R. D. Yates, and M. Gruteser, "Real-time status: How often should one update?" in *2012 Proceedings IEEE INFOCOM*. Orlando, FL, USA: IEEE, 2012, pp. 2731–2735.

[16] K. Nichols and V. Jacobson, "Controlling queue delay," *Communications of the ACM*, vol. 55, no. 7, pp. 42–50, Jul. 2012.

[17] M. Shreedhar and G. Varghese, "Efficient fair queueing using deficit round robin," in *Proceedings of the ACM SIGCOMM '95 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*. Cambridge, MA, USA: ACM, 1995, pp. 231–242.

[18] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proceedings of the 2016 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2016, pp. 3464–3468.

[19] G. Bernat, A. Burns, and A. Llamosí, "Weakly hard real-time systems," *IEEE Transactions on Computers*, vol. 50, no. 4, pp. 308–321, Apr. 2001.

[20] M. Hamdaoui and P. Ramanathan, "A dynamic priority assignment technique for streams with (m, k)-firm deadlines," *IEEE Transactions on Computers*, vol. 44, no. 12, pp. 1443–1451, Dec. 1995.

[21] M. Li, Y.-X. Wang, and D. Ramanan, "Towards streaming perception," in *Computer Vision – ECCV 2020*, ser. Lecture Notes in Computer Science, vol. 12347. Springer, 2020, pp. 473–488.

[22] J. Yang, S. Liu, Z. Li, X. Li, and J. Sun, "Real-time object detection for streaming perception," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 5375–5385.

[23] W. Jo, K. Lee, J. Baik, S. Lee, D. Choi, and H. Park, "DaDe: Delay-adaptive detector for streaming perception," in *Proceedings of the 18th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2023) – Volume 5: VISAPP*. SciTePress, 2023, pp. 39–46.

[24] A. Ghosh, V. Balloli, A. Nambi, A. Singh, and T. Ganu, "Chanakya: Learning runtime decisions for adaptive real-time perception," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/ae2d574d2c309f3a45880e4460efd176-Abstract-Conference.html

[25] J. Lee, B. Varghese, and H. Vandierendonck, "ROMA: Run-time object detection to maximize real-time accuracy," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023, pp. 6394–6403.

[26] J. Peng, T. Wang, J. Pang, and Y. Shen, "Towards latency-aware 3D streaming perception for autonomous driving," 2025. [Online]. Available: https://arxiv.org/abs/2504.19115

[27] I. Gog, S. Kalra, P. Schafhalter, J. E. Gonzalez, and I. Stoica, "D3: A dynamic deadline-driven approach for building autonomous vehicles," in *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22)*. Rennes, France: ACM, 2022, pp. 453–471.

[28] L. Liu, Z. Dong, Y. Wang, and W. Shi, "Prophet: Realizing a predictable real-time perception pipeline for autonomous vehicles," in *Proceedings of the 2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 305–317.

[29] S. Liu, X. Fu, M. Wigness, P. David, S. Yao, L. Sha, and T. Abdelzaher, "Self-cueing real-time attention scheduling in criticality-aware visual machine perception," in *Proceedings of the 2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2022, pp. 173–186.

[30] Y. Shi, S. Qayyum, S. A. Memon, U. Khan, J. Imtiaz, I. Ullah, D. Dancey, and R. Nawaz, "A modified Bayesian framework for multi-sensor target tracking with out-of-sequence-measurements," *Sensors*, vol. 20, no. 14, p. 3821, 2020.

[31] V. Nigade, P. Bauszat, H. E. Bal, and L. Wang, "Jellyfish: Timely inference serving for dynamic edge networks," in *2022 IEEE Real-Time Systems Symposium (RTSS)*. Houston, TX, USA: IEEE, 2022, pp. 277–290.

[32] Z. Zhang, Y. Zhao, H. Li, and J. Liu, "BCEdge: SLO-aware DNN inference services with adaptive batch-concurrent scheduling on edge devices," *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4131–4145, 2024.

[33] N. Tatbul and S. B. Zdonik, "Window-aware load shedding for aggregation queries over data streams," in *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB '06)*. Seoul, Korea: VLDB Endowment, 2006, pp. 799–810. [Online]. Available: https://dl.acm.org/doi/10.5555/1182635.1164196

[34] B. Maurer, "Fail at scale: Reliability in the face of rapid change," *Communications of the ACM*, vol. 58, no. 11, pp. 44–49, Nov. 2015.

[35] R. D. Yates, Y. Sun, D. R. Brown, S. K. Kaul, E. Modiano, and S. Ulukus, "Age of information: An introduction and survey," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 5, pp. 1183–1210, May 2021.

[36] J. W. S. Liu, K.-J. Lin, W.-K. Shih, A. C.-s. Yu, J.-Y. Chung, and W. Zhao, "Algorithms for scheduling imprecise computations," *Computer*, vol. 24, no. 5, pp. 58–68, May 1991.

[37] S. Yao, Y. Hao, Y. Zhao, H. Shao, D. Liu, S. Liu, T. Wang, J. Li, and T. Abdelzaher, "Scheduling real-time deep learning services as imprecise computations," in *2020 IEEE 26th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2020, pp. 1–10.

[38] Y. Li, A. Padmanabhan, P. Zhao, Y. Wang, G. H. Xu, and R. Netravali, "Reducto: On-camera filtering for resource-efficient real-time video analytics," in *Proceedings of the 2020 Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '20)*. Virtual Event, USA: ACM, 2020, pp. 359–376.

[39] J. Jiang, G. Ananthanarayanan, P. Bodík, S. Sen, and I. Stoica, "Chameleon: Scalable adaptation of video analytics," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18)*. Budapest, Hungary: ACM, 2018, pp. 253–266.

[40] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia, "Noscope: Optimizing neural network queries over video at scale," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1586–1597, 2017.

[41] M. A. Arefeen, S. T. Nimi, and M. Y. S. Uddin, "Framehopper: Selective processing of video frames in detection-driven real-time video analytics," in *Proceedings of the 18th International Conference on Distributed Computing in Sensor Systems (DCOSS)*. Marina del Rey, CA, USA: IEEE, 2022, pp. 125–132.